

Desenvolvimento e Análise de um Simulador de Circuitos Quânticos

Tallys Assis de Souza, Henrique Cota de Freitas

Instituto de Ciências Exatas e Informática – Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

tallys.assis@sga.pucminas.br, cota@pucminas.br

Abstract. *This work addresses the exploration and implementation of quantum circuit simulators, aiming to make quantum computing more accessible and for students and enthusiasts. The research aims to fill the educational gap in understanding the fundamental concepts of quantum computing and practical experimentation with quantum circuits. This work provides a comprehensive guide to facilitate learning and experimentation in this field.*

Resumo. *Este trabalho aborda a exploração e implementação de simuladores de circuitos quânticos, visando tornar a computação quântica mais acessível para estudantes e entusiastas. A pesquisa visa preencher a lacuna educacional existente na compreensão dos conceitos fundamentais da computação quântica e na experimentação prática com circuitos quânticos. Este trabalho fornece um guia abrangente para facilitar a aprendizagem e a experimentação nesse campo.*

1. Introdução

A computação quântica é um campo em rápido desenvolvimento que oferece promessas empolgantes para o futuro da tecnologia da informação. Um dos maiores campos de trabalho dentro dessa área é o de implementação de simuladores confiáveis para substituir os computadores físicos quando estes não estão disponíveis [Viamontes et al., 2009]. Isto é especialmente importante para testes menores ou provas de conceito que não necessitam de uma instalação muito grande, e não precisam ocupar necessariamente o tempo em um computador quântico físico.

A principal vantagem de um simulador para um computador físico, é que para escalas consideravelmente menores qualquer pessoa pode executar testes e circuitos em suas próprias máquinas, e em escalas médias, empresas ou organizações ainda podem dedicar hardware para estas tarefas sem a necessidade de recorrer a todo momento a um computador convencional fornecido por outra entidade.

Dentro do campo de simuladores, pouco material é dedicado à educação. A grande parte das publicações são relacionadas à computação quântica, em geral, mas

pouco dedicadas para técnicas de simulação voltados para o aprendizado. Mesmo sendo um campo com uma previsão de substituição pelo hardware quântico a medida que os computadores físicos se tornam mais acessíveis, os simuladores vão continuar sendo ferramentas importantes demais para serem deixadas de lado. O principal desafio é capacitar desenvolvedores familiarizados com essa área para desenvolver simuladores de nível comercial que satisfaçam requisitos de pesquisas de pequena e média escala. Nesse caso, como capacitar um iniciante a trabalhar com essa área?

O objetivo principal é desenvolver um conjunto de materiais teóricos e ferramentas práticas que capacitem estudantes e entusiastas a compreenderem e explorarem a simulação de circuitos quânticos. Estes recursos visam preencher a lacuna educacional existente e tornar a computação quântica mais acessível, possibilitando que os estudantes, profissionais e entusiastas possam trabalhar em projetos de computação quântica sem a dependência de um computador físico.

Para alcançar o objetivo principal, este estabelece os seguintes objetivos:

- Desenvolver um simulador base que encapsule os princípios básicos da computação quântica e da simulação de circuitos quânticos.
- Criar uma interface de uso intuitiva que ajude os usuários a se familiarizar com soluções atuais.
- Fornecer uma análise comparativa com outras ferramentas existentes na área da computação quântica.
- Explorar possíveis melhorias e otimizações para as técnicas de simulação apresentadas.

Com este trabalho, espera-se prover um material de introdução para estudantes e entusiastas que desejam ingressar na área de computação quântica, e aprender na prática a arquitetura lógica e o funcionamento interno de um computador quântico além das abstrações com as fornecidas por uma API.

Este artigo está organizado da seguinte forma:

Seção 2 apresenta uma revisão da literatura e introdução ao conteúdo, seção 3 apresenta a metodologia e os resultados esperados, seção 4 apresenta um guia de implementação e discussões sobre técnicas usadas da seção 3, seção 5 apresenta uma análise da implementação abordada e também comparações com plataformas consolidadas com os resultados obtidos, por fim, uma seção de conclusões.

2. Fundamentos Teóricos de Computação Quântica e Simulação de Sistemas Quânticos

Sendo este um trabalho de cunho didático, o repertório bibliográfico se concentra em

referenciais teóricos mais do que trabalhos correlatos, mas não os descartando na totalidade.

É importante compreender os conceitos que estruturam toda a lógica dos sistemas quânticos mais abstratos, como representações em espaços vetoriais e sistemas lineares complexos, e só então os aplicar em um campo prático. Para isso, a estrutura lógica abordada é dividida em três camadas exploradas a seguir: abstração de sistemas físicos, programação de computadores quânticos e simulação.

2.1. A Abstração dos Sistemas Quânticos

Sistemas de computação quântica, no escopo físico, diferem consideravelmente dos conceitos usados no escopo teórico, que por sua vez diferem dos conceitos usados no escopo simulado.

Sistemas físicos utilizam representações problemáticas para uso dentro do campo de computação, muito dependentes de representações como funções de onda muito pesadas para serem computadas com precisão sem um custo muito alto em desempenho e energia, tornando a elaboração de soluções dentro dessa abordagem consideravelmente mais complexas.

Transportar a lógica física para o campo teórico permite estruturar conceitos de forma sólida, abstraindo sistemas em física quântica pura para sistemas lineares [Nielsen et al., 2000] que usam uma lógica matemática mais convencional [Sobral & Machado, 2019]. A mais popular dessas representações é o vetor de estados representado por vetores em um espaço de Hilbert, que é um espaço vetorial complexo.

Saindo do campo físico é possível dedicar esforços ao escopo prático da computação quântica e trabalhar em um campo de estudos mais restrito [Nannicini, 2020], facilitando o entendimento dos sistemas quânticos, em geral [Khalid, 2012] [Charemza, 2005][Rieffel & Polak, 2011].

O *qubit* físico não possui uma implementação padrão. Alguns laboratórios os implementam com supercondutores, outros com nitrogênio e diamante e também há algumas implementações com pontos quânticos. Para a abstração matemática é normalmente utilizada uma abstração do modelo de semicondutores, pois a interação em ambos os modelos é mais parecida e simples de levar de um modelo a outro.

Na representação matemática, cada *qubit* é visualmente interpretado como uma esfera de Bloch. A esfera de Bloch, Figura 1, é uma representação tridimensional que interpreta o estado do *qubit* como um vetor que parte do centro para a superfície. O polo norte da esfera representa o estado 0, o polo sul o estado 1, e todo o espaço intermediário representa diversas superposições possíveis para o *qubit*.

Nessa representação, operações no *qubit* são representadas como rotações do vetor em vários eixos.

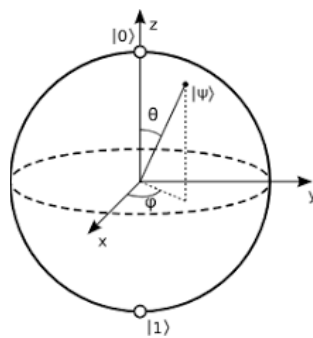


Figura 1. Diagrama da esfera de Bloch
Fonte: Wikimedia Foundation

Matematicamente, o estado do *qubit* é representado como uma combinação de probabilidades para cada estado que devem se somar em 1. Para facilitar interações entre múltiplos *qubits*, as probabilidades cruas são substituídas por sua raiz quadrada na forma de números complexos.

Essa representação exhibe as principais diferenças dos *qubits* para *bits* convencionais. Invés de assumir valores discretos de 0 e 1, um *qubit* tem um campo virtualmente infinito de estados intermediários que guardam muito mais informação em um único elemento.

Dentro do modelo abstraído, podemos interpretar cada qubit como uma combinação linear de dois estados $|0\rangle$ e $|1\rangle$ [Nielsen et al., 2000]. Cada um dos estados está associado a um coeficiente chamado de amplitude de probabilidade, que é um número complexo que representa a raiz quadrada da probabilidade do qubit estar naquele estado quando observado. Por exemplo: um qubit no estado $\sqrt{1/2} |0\rangle + \sqrt{1/2} |1\rangle$ tem 50% de chance de ser observado em cada estado. Essa situação é resumidamente como funciona a ideia de superposição de estados dentro de circuitos quânticos. No mundo físico a partícula simulada estaria em um estado onde se comporta como qualquer um dos dois estados, até que alguma interação a force a colapsar para qualquer um dos estados possíveis.

Na lógica de sistemas lineares, as operações em *qubits* são descritas por matrizes unitárias chamadas portas quânticas. Essas portas manipulam a amplitude das probabilidades associadas aos estados quânticos conservando sua proporção. Diferentes portas realizam diferentes transformações nos *qubits* como rotações. Propriedades mais complexas, como o entrelaçamento, onde o estado de um qubit depende do estado de outro, podem ser descritas por esse sistema com portas maiores. Cada porta é representada por uma matriz de $N \times N$ dimensões onde N é o número de estados afetados por ela.

2.2. Programação de Computadores Quânticos

A estrutura de projetos quânticos difere bastante da lógica imperativa mais comum em sistemas computacionais, seguindo uma estrutura mais declarativa que se assemelha às

linguagens HDL de descrição de hardware em certo nível [Schumacher, Benjamin, 1993][Eric et al., 2019], e por isso sistemas de computação quântica são chamados circuitos quânticos.

Sistemas quânticos são representados por portas lógicas de natureza particular a esses sistemas, atuando como transformações lineares nos estados dos *qubits* na maioria das abstrações utilizadas. Se observarmos o espaço linear de um qubit como um espaço 3d, podemos também visualizar essas operações como rotações nesse espaço, e calcular seu resultado a partir disso.

A descrição destes sistemas pode ser interpretada como uma sequência de operações individuais, mais do que instruções imperativas convencionais em linguagens como C [Johnston et al., 2019][Yanofsky & Mannucci, 2008].

2.3. Trabalhos Correlatos em Simulação de Sistemas Quânticos

As abstrações matemáticas de sistemas de computação quântica geralmente podem ser traduzidas para a computação convencional sem alterações bruscas [Viamontes et al., 2009], porém muitos aspectos são inviáveis devido aos limites atuais e conceituais dos próprios computadores [Kelly, 2018], principalmente no que diz respeito ao consumo de memória, exigindo soluções mais elaboradas a fim de contornar os problemas encontrados.

O principal desses problemas está na aplicação das operações das portas lógicas. Uma vez que tratamos sistemas quânticos com lógica de álgebra linear, operações são representadas como transformações lineares por meio de multiplicação de matrizes. Isso implica que uma matriz deve ter dimensões equivalentes ao vetor dos estados de *qubits*, o que leva o tamanho de memória necessário para comportar essas matrizes a números inviáveis na ordem de uma função exponencial. É necessário então reduzir essas operações ao seu nível elementar e computar a partir disso [Kelly, 2018].

A natureza das abstrações de sistemas lineares tornam sistemas quânticos favoráveis a sistemas modulares e paralelos, seja em software ou em hardware [Khalid, 2005]. Aplicações de portas quânticas, quando previamente processadas, são representadas por uma multiplicação de matrizes simples, que pode ser mais simplificada em operações elementais executadas em paralelo por todo o vetor de estados, sem um grande risco de conflitos de memória. Portas mais simples como X, Y e Z podem ser otimizadas com aceleração por GPUs, devido à alta capacidade de cálculos de ponto flutuante que executam.

Abrindo mão parcialmente da precisão dentro do ambiente simulado contornando processos mais abstratos é possível os encapsular em ambientes menos convencionais para projetos dessa natureza, sem comprometer os resultados e capacitando o simulador a ser executado em sistemas mais modestos [Wang, 2019].

A técnica mais convencional para expandir o produto de matrizes e evitar o

consumo exponencial de memória, é separar os *qubits* em grupos e processar a aplicação das portas em si em paralelo, oferecendo um ganho de desempenho considerável principalmente em dispositivos altamente paralelos como FPGAs e GPUs via plataformas como CUDA e OpenCL; e CPUs especializadas como as usadas em servidores[Kelly, 2018]. Como o tamanho do vetor de operações será igual a *Qubits* / Tamanho da porta, uma execução em paralelo pode uma melhoria considerável em comparação com a execução sequencial, com ganhos consideravelmente maiores com dispositivos mais especializados como FPGAs e GPUs, contanto que o acesso ao vetor de memória possa ser feito em paralelo.

Uma técnica de simplificação para simuladores é se aproveitar da propriedade de universalidade dos circuitos quânticos. A universalidade aponta que dada a implementação de algumas portas quânticas, as outras podem ser implementadas como combinações de portas mais básicas, assim como a propriedade da porta NAND em computadores convencionais. Isso permite aos desenvolvedores focar esforços para suportar apenas o necessário para ter um computador capaz de executar qualquer algoritmo quântico.

3. Metodologia e Resultados Esperados

A seção de Metodologia proporciona uma visão detalhada dos passos cruciais adotados durante o desenvolvimento do simulador de circuitos quânticos. Aqui, serão delineados os materiais utilizados, o método empregado e o cronograma estabelecido para guiar o progresso do trabalho.

3.1. Materiais Utilizados e Fontes Consultadas

Nesta fase do desenvolvimento, uma cuidadosa seleção de materiais teóricos e práticos foi realizada para fundamentar e aplicar os conceitos de simulação de circuitos quânticos. A base teórica foi construída principalmente a partir de fontes mais conhecidas, como os livros "Quantum Computation and Quantum Information," [Nielsen et al., 2000] "Quantum Circuit Simulation," [Viamontes et al., 2009] e "Programming Quantum Computers" [Eric et al., 2019].

No âmbito prático, a implementação se beneficiou de uma abordagem multifacetada. Utilizando-se de uma implementação existente de simulador como referência, foram também incorporados dois projetos de simulação, ambos implementam técnicas de agrupamento que foram reimplementadas convencionalmente no projeto final. Essas escolhas foram orientadas pela necessidade de validar teoricamente os resultados e, simultaneamente, explorar implementações concretas para aprimorar a compreensão prática do tema.

3.2. Método

O desenvolvimento do simulador foi dividido em 3 grandes etapas, sendo elas o estudo e prototipagem, a implementação e a validação.

A etapa de estudo e prototipagem consistiu basicamente em construir um esqueleto base para o simulador e estudar formas de contornar os problemas de memória causados pelas operações de portas. Para contornar o gasto exponencial de memória foi usada uma técnica alternativa aos produtos tensores, dismantelando as operações em uma estrutura estática que não escalona o gasto de memória.

A implementação teve de ser dividida em 4 etapas, cada uma com cerca de duas semanas conforme a descrição a seguir.

Inicialmente foi implementado um computador estático para validar as representações de *qubits* e as capacidades e limites dos tipos usados no vetor de amplitudes, implementado com a classe `std::complex` em C++ que pode representar as amplitudes complexas dos estados.

A segunda e mais complexa fase foi a de implementação dos algoritmos de aplicação de portas, adaptados para portas unitárias e portas de dois *qubits*. As funções auxiliares para o devido funcionamento das funções de aplicação também foram implementadas em separado.

Uma API de código mais limpa foi desenvolvida para facilitar o uso, encapsulando as portas como constantes em funções separadas. O código do simulador também foi refatorado nessa etapa para diluir o código e mais arquivos, e também as funções de medição e colapso.

Para tornar o uso mais acessível, a interface de utilização foi projetada para se aproximar de soluções mais famosas como o Qiskit em uma API de código.

3.3. Portas Implementadas

O simulador implementa diversas portas, mas aproveita a propriedade de universalidade para abdicar da implementação de portas mais complexas. As portas implementadas nativamente no simulador são:

Porta identidade que não afeta *qubits*. É representada pela matriz

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Porta Pauli-X ou NOT que inverte os estados de um *qubit*. A porta X na representação de Bloch rotaciona o vetor do estado no eixo X em 180 graus

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Porta Pauli-Y que performa a mesma operação da porta X, porém rotaciona o

estado no eixo Y. Na interpretação matemática, ele rotaciona o vetor no eixo imaginário, modificando valores i do vetor.

$$\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

Porta Pauli-Z que performa a rotação no eixo Z realizando uma rotação de fase.

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Porta Hadamard que aplica uma superposição simétrica ao *qubit*. Em um *qubit* no estado $|0\rangle$, ele ficaria na superposição simétrica entre $|0\rangle$ e $|1\rangle$.

$$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Porta T que aplica uma rotação de 45 graus no eixo Z realizando uma rotação de fase. A porta T conjugada também foi implementada e desfaz a operação da porta T.

$$T = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix} \quad T^\dagger = \begin{bmatrix} 1 & 0 \\ 0 & e^{-i\pi/4} \end{bmatrix}$$

Porta de fase S que realiza uma rotação de 90 graus no eixo Z bem como sua conjugada.

$$S = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix} \quad S^\dagger = \begin{bmatrix} 1 & 0 \\ 0 & -i \end{bmatrix}$$

Porta NOT controlada, CNOT ou CX. A porta CNOT é uma porta X que tem seu efeito vinculado ao estado de um qubit de controle. Quando o qubit de controle está no estado 0, o qubit alvo terá seus estados preservados, e quando o qubit de controle estiver no estado 1, o qubit alvo terá uma aplicação de X.

Nota-se que por afetar mais qubits, a matriz de operação é proporcionalmente maior, iterando com o vetor de estados de 2 qubits, que corresponde a quatro estados.

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

É importante considerar que a interpretação das ações das portas sobre os *qubits* é referenciada como operações geométricas na esfera de Bloch. Esta interpretação não só simplifica o entendimento como permite uma interpretação visual do *qubit*.

Portas mais complexas podem ser implementadas como combinações de portas mais simples pela propriedade de universalidade.

4. Simulador Quantum Console

4.1. Objetivo da Implementação

O objetivo da implementação do simulador é garantir precisão de resultados ao mesmo tempo que evita tornar a estrutura do código extremamente complexa. Para isso várias abordagens para abstração foram evitadas em favor de uma implementação mais direta.

4.2. Arquitetura do Projeto

A estrutura do simulador consiste em uma classe do simulador isolada como *back-end* e uma interface de programa para uso. Nenhuma biblioteca fora o conjunto padrão das bibliotecas C++ para sistemas Linux foi usada.

O simulador inicializa um computador virtual, e executa as operações com adição de portas. Os cálculos são feitos durante a adição de portas, e o colapso calculado posteriormente.

Para legibilidade, a classe do simulador foi dividida em vários arquivos de implementação sob um único *header* com as declarações. Essa abordagem evita arquivos muito grandes ao mesmo tempo que cria uma estrutura lógica das ferramentas do simulador.

A utilização do simulador implementa uma interação onde o usuário inicializa um computador com N *qubits*, aplica várias portas e sorteia um estado do vetor de estados no momento da medição de acordo com suas probabilidades. Dessa forma as operações são aplicadas em tempo de aplicação, acelerando a performance de medição.

4.3. Ferramentas de Implementação

O projeto foi feito na IDE Qt Creator, com estrutura padrão do modelo Cmake, possibilitando portabilidade de plataforma. A linguagem de escolha foi C++, garantindo maior flexibilidade no controle de memória e eficiência de otimizações em tempo de compilação. Cada iteração do simulador foi testada com a compilação de menor tamanho de binário fornecida pelo Cmake.

4.4. Modelagem de Sistemas Quânticos

O principal elemento do simulador é o vetor de estados, declarado como um `std::vector` de valores complexos usados da classe `std::complex`. O vetor é inicializado como $2^{\text{número de qubits}}$, para mapear a quantidade certa de estados.

A aplicação de portas utiliza a abordagem de agrupamento usada em outros projetos [Kelly, 2018][Wang, 2019]. A adição de portas é mapeada por funções auxiliares que invocam a aplicação com sua respectiva matriz acompanhada dos *qubits* afetados. Duas implementações de aplicação foram feitas para acomodar a porta CNOT

que requisita 2 *qubits*.

A paralelização pôde ser implementada com segurança na aplicação de portas, com a iteração no vetor de *qubits* podendo ser executada em paralelo atentando ao acesso ao vetor em si que é uma zona crítica. Em uma implementação usando um vetor simples invés de uma classe vector possibilitaria o acesso paralelo ao vetor eliminando zonas críticas, porém aumentando consideravelmente a complexidade do código devido às validações de acesso necessárias para evitar erros.

A implementação de um sistema de cache de agrupamento de *qubits* não resultou em melhoras. A importação do cache em uma lista tem complexidade $O(N)$ similar ao cálculo em tempo de execução.

Com portas o suficiente para atingir universalização e com a adição de portas mais comuns, não se viu necessidade de implementar diretamente portas mais complexas como Toffoli que tornariam o código mais complexo que o proposto.

4.5. Validação

A validação inicial do simulador foi feita replicando circuitos de teste entre o simulador implementado e a plataforma IBM quantum. A plataforma foi escolhida como referência por ser mais acessível e ser uma ferramenta já usada em produção. O desempenho do simulador foi medido com um *benchmark* de um circuito com 10 portas e 5 *qubits*. Para efeitos de padronização, todas as medições foram feitas em uma máquina física com processador Ryzen 5 3500U, com 8GB de RAM executando o sistema operacional KDE Neon com kernel Linux 6.2. O objetivo do *benchmark* foi medir os ganhos de performance com diferentes otimizações.

4.6. Melhorias de Performance

Todavia a execução em paralelo para a maioria dos circuitos não apresenta ganho, pois o *overhead* gerado abate o tempo ganho com a aplicação de portas. Esse problema afeta computadores com *clocks* mais rápidos com uma execução em *underclock* atingindo melhores ganhos. Uma reimplementação com um mapeamento de memória direto no lugar de um objeto vetor removeria as zonas críticas e garantiria um *speedup* melhor, porém a complexidade adicionada ao código torna mais atrativo abrir mão da performance em nome da legibilidade, dado o objetivo do simulador.

Estima-se que para máquinas modernas o ganho seja dispensável pois otimizações de acesso à memória e cálculo de aritmética de ponto flutuante oferecem maior ganho de desempenho.

Uma abordagem para reduzir o tempo de execução seria implementar um cache dos agrupamentos de *qubits* para determinados inputs. Assim seria possível pular etapas de agrupamento economizando tempo a medida que mais grupos fossem armazenados. Porém, sem um *cast* direto para memória seria impossível obter melhora com um

processamento em *loop* tendo a mesma complexidade do processamento padrão.

4.7. Limitações

O simulador aparenta cometer erros quando operando com altos números de *qubits* devido a erros de endereçamento com vetores muito grandes, indicando uma limitação da arquitetura em si. O consumo de memória também é uma preocupação relevante, uma vez que o vetor de estados dobra para cada qubit adicionado. Com uma grande margem, é seguro afirmar que o consumo de memória é o maior gargalo do simulador, demandando uma grande quantidade disponível até que a velocidade de processamento seja gravemente prejudicada pelo tamanho das iterações.

4.8. Distribuição

O projeto está hospedado publicamente como código-fonte sob a licença GNU GPL-3.0 no repositório do GitHub: <https://github.com/Talkys/Quantum-Console>

5. Resultados

5.1. Validação de Resultados

O simulador se mostrou capaz de manter resultados colapsados comparáveis ao IBM Quantum sem desvios maiores. Porém, dada a limitação de precisão de ponto flutuante da CPU x86 em si, não se pode garantir precisão de alta qualidade comparado com um simulador quântico comercial com correções como o Quiskit.

Para um circuito de 6 *qubits* com portas H aplicadas nos *qubits* 1, 3 e 5 são esperados resultados balanceados para as saídas em decimal 0, 2, 8, 10, 32, 34, 40 e 42. Para condensar os resultados foram realizadas 10000 iterações para validar as saídas.

Como a saída deve ser igualmente balanceada, espera-se que cada resultado em uma amostra ideal apareça em 12,5% dos casos, correspondente a um oitavo do total de medições. O resultado real pode ser observado na Tabela 1.

Resultado	Saída Quantum Console	Saída IBM Quantum
000000 (0)	10,6%	12,7%
000010 (2)	13,0%	13,3%
001000 (8)	12,8%	13,4%
001010 (10)	13,5%	11,8%
100000 (32)	12,3%	12,3%
100010 (34)	13,7%	11,8%
101000 (40)	11,3%	13,3%
101010 (42)	12,8%	11,2%

Tabela 1. Comparação de resultados com os simuladores IBM
Fonte: Autor

O simulador da plataforma IBM Quantum teve um desvio menor, porém os resultados são próximos o bastante, com o Quantum Console mantendo mais resultados perto do ideal 12,5% para cada saída.

5.2. Avaliação de Performance

Para medir a performance do simulador, um *benchmark* com o circuito a seguir foi implementado pela API e executado em 100, 10000 e 100000 iterações.

O circuito de testes é um circuito simples, Figura 2, sem propósito computacional projetado para estressar a máquina e tirar medições. As portas CNOT usadas não tem efeito prático e as Hadamards aplicadas duas vezes em sequência anulam o resultado para 0.

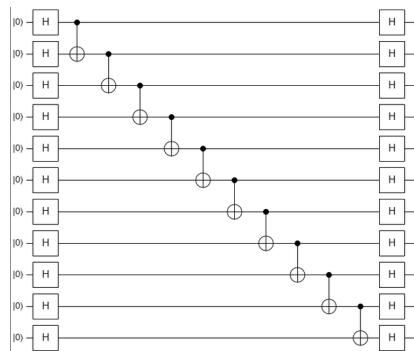


Figura 2. Circuito usado no benchmark
Fonte: Autor

A paralelização foi um incremento funcional do simulador, sem aumentar a complexidade do código, e garantindo um *speedup* de até 25% em casos muito extremos.

A execução usando *threads* não físicas como em uma CPU 4/4 resultou em um aumento do tempo de execução com o *overhead* das *threads* sobrepondo a execução do programa em si.

Os resultados compilados foram agrupados como *threads* x iterações, Tabela 2.

	100 Iterações	1000 Iterações	10000 Iterações
Single Thread	1,20894s	12,1384s	87,8169s
2 Threads	0,977323s	9,84173s	85,6604s
4 Threads	0,9722488s	9,76108s	77,1343s
8 Threads	1,47291s	13,7018s	105,827s

Tabela 2. Tempos de execução para várias configurações de *threads* e iterações

Fonte: Autor

6. Conclusão

Este trabalho teve como objetivo principal desenvolver um simulador de circuitos quânticos que fosse acessível e didático para estudantes e entusiastas da computação quântica. Para isso, foram delineados e alcançados vários objetivos específicos, como a criação de um simulador base que encapsulasse os princípios fundamentais da computação quântica e a realização de uma análise comparativa com outras ferramentas existentes na área. A abordagem metodológica incluiu etapas de estudo e prototipagem, implementação e validação, garantindo um desenvolvimento estruturado e fundamentado.

Os resultados obtidos demonstraram que o simulador desenvolvido, denominado Quantum Console, é capaz de manter resultados colapsados comparáveis aos obtidos pelo IBM Quantum, ainda que apresente algumas limitações em termos de precisão de ponto flutuante e consumo de memória. A validação dos resultados mostrou que, para circuitos modestos, os desvios observados foram mínimos, com o Quantum Console apresentando uma distribuição de resultados próxima do ideal. Além disso, a avaliação de performance revelou que o simulador é capaz de executar circuitos quânticos com eficiência, embora o consumo de memória ainda represente um gargalo significativo para a escalabilidade do sistema.

Para trabalhos futuros, recomenda-se explorar técnicas de otimização que possam reduzir o consumo de memória e melhorar a precisão dos cálculos, como a utilização de GPUs e FPGAs para processamento paralelo. Além disso, o desenvolvimento de uma interface gráfica (GUI) mais intuitiva e funcional poderia tornar o simulador ainda mais acessível e útil para a comunidade acadêmica e para os

desenvolvedores. Outras melhorias poderiam incluir a implementação de novos algoritmos quânticos e a ampliação das capacidades de simulação para suportar um número maior de qubits, potencializando assim a utilidade do Quantum Console como uma ferramenta educacional e de pesquisa.

Referências

Nielsen, M. A., & Chuang, I. L. (2000). Quantum Computation and Quantum Information. Cambridge University Press.

Viamontes, G. F., & Markov I. L., & Hayes J. P. (2009). Quantum Circuit Simulation. Springer Dordrecht Heidelberg London New York

Eric R. Johnston, & Nic Harrigan, & Mercedes Gimeno-Segovia (2019) Programming Quantum Computers. O'Reilly Media

Schumacher, Benjamin. (1993). Quantum coding. Department of Physics, Kenyon College,

Charemza, Michhal (2005). An Introduction to Quantum Computing. University of Warwick.

Khalid, Abdullah (2012). A Gentle Introduction to Quantum Computing. Lahore University of Management Sciences.

Nannicini Giacomo (2020). An Introduction to Quantum Computing, Without the Physics. IBM T.J Watson.

Sobral, João B. M. & Machado, Renato B. (2019). Computação Quântica: Aspectos Físicos e Matemáticos – Uma Abordagem Algébrica. Universidade Federal de Santa Catarina.

Johnston, E. R., Harrigan, N., & Gimeno-Segovia, M. (2019). Programming Quantum Computers. O'Reilly Media.

Rieffel, E., & Polak, W. (2011). Quantum Computing: A Gentle Introduction. The MIT Press.

Yanofsky, N. S., & Mannucci, M. A. (2008). Quantum Computing for Computer Scientists. Cambridge University Press.

Khalid, Ahmed U. (2005). FPGA Emulation of Quantum Circuits. Department of Electrical & Computer Engineering, McGill University

Kelly, Adam (2018). Simulating Quantum Computers Using OpenCL. <https://qcgpu.github.io/>

Wang, Yiyi (2019). qubit.js. <https://github.com/shd101wyy/qubit.js>

IBM Quantum em <https://www.ibm.com/quantum>